

Listening Data

Data Product end-to-end: pipeline ETL, modelado dimensional, enriquecimiento geográfico y exploración analítica interactiva.

VERSIÓN	4.0 (reestructurada)
ÚLTIMA REVISIÓN	Septiembre 2026
AUTOR	Pedro Parada
ROL	Data Engineer / Backend Developer
ESTADO	Documentación técnica — Portafolio público

ENFOQUE GENERAL

Un Data Product, para decisiones Data-Driven

Este documento presenta el proyecto como un **Data Product / End-to-End Data Platform**: un conjunto de datos tratado como un producto en sí mismo, con un pipeline reproducible, un modelo de datos explícito y consumidores bien definidos, en vez de un análisis puntual o un script aislado. Para reflejar eso con claridad, el proyecto se separa explícitamente en tres capas.

Data Engineering	ETL, transformación, enriquecimiento, calidad y modelado. Es la capa más madura y completamente implementada del proyecto.
Backend / Software	Arquitectura del pipeline, contratos de datos entre sus componentes y las decisiones de diseño detrás de cada elección técnica.
Data Analytics	Métricas, visualización, BI e insights, implementada principalmente a través del frontend interactivo (opcional) y de la integración con Power BI.

Cada capa tiene su propio nivel de madurez y su propia sección dedicada, de modo que el lector pueda entender rápidamente qué parte del proyecto es el núcleo (Data Engineering), cuál es la arquitectura que lo sostiene (Backend) y cuál convierte los datos en decisiones (Analytics).

Índice

01	Overview	4
02	Problem & Objectives	6
03	Data Engineering	7
04	Data Architecture & Modeling	9
05	Backend Engineering	11
06	Frontend & Data Visualization	13
07	Testing & Data Quality	14
08	Engineering Decisions / ADRs	15
09	Reproducibility	17
10	Project Structure	18

1 Overview

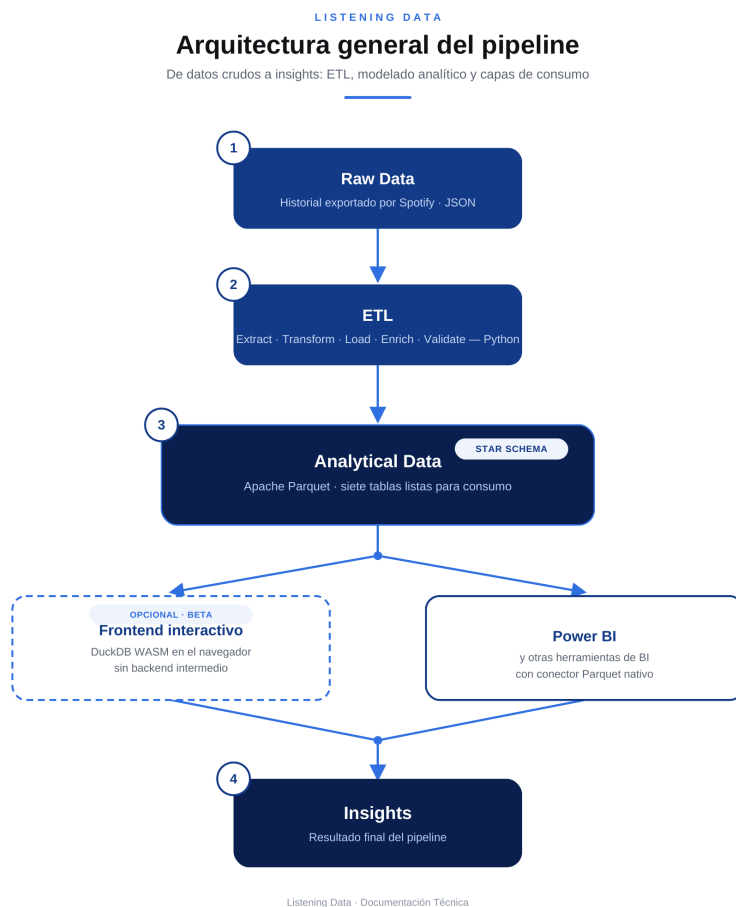
Qué es el proyecto. Una plataforma de datos que transforma el historial de reproducción exportado por Spotify (JSON) en un dataset analítico modelado en esquema estrella y persistido en Apache Parquet, con una capa opcional de exploración interactiva en el navegador.

Problema que resuelve. El export de Spotify entrega datos crudos, semi-estructurados y sin modelar, pensados para portabilidad personal y no para análisis. No hay dimensiones reutilizables, no hay contexto geográfico de las IPs registradas, y el formato no es eficiente para herramientas de BI.

Datos utilizados. Archivos JSON del historial de reproducción exportado por Spotify (eventos de tracks y episodios, con metadata de dispositivo, timestamp e IP).

Qué produce. Siete archivos Parquet (una tabla de hechos y cinco dimensiones, más una dimensión geográfica enriquecida) listos para consumo analítico, y opcionalmente una aplicación web de exploración.

Arquitectura general (alto nivel)



Tanto el frontend interactivo como las herramientas de BI consumen los archivos Parquet directamente, cada uno mediante su propio motor de consulta: DuckDB WASM en el navegador para el frontend, o el conector Parquet nativo en el caso de las herramientas de BI.

Stack tecnológico (resumen). Python, pandas, PyArrow, GeoIP2/MaxMind GeoLite2 y pytest para la capa de datos; React, TypeScript, Vite, DuckDB WASM y Apache ECharts para el frontend opcional.

Estado actual

Capa	Estado
Data Engineering (ETL, modelado, calidad)	Implementado y documentado
Enriquecimiento geográfico (GeoLite2)	Implementado — fallos no fatales
Backend Engineering (pipeline modular)	Implementado y documentado
Frontend interactivo	Implementado — beta
Integración BI	Implementado — vía Parquet + Power BI

2

Problem & Objectives

Contexto

Spotify permite a sus usuarios solicitar y exportar su historial de reproducción en formato JSON. Es un formato pensado para portabilidad de datos personales, no para análisis: no tiene estructura dimensional, mezcla eventos de música y de podcasts, y no incluye contexto geográfico más allá de un código de país y una IP.

Problema

Los datos raw presentan limitaciones concretas para el análisis:

- Estructura plana, sin dimensiones reutilizables ni claves subrogadas.
- Columnas de audiobooks frecuentemente vacías.
- Timestamps en formato de texto en vez de tipos temporales explícitos.
- IPs sin contexto geográfico (ciudad, región, ISP).
- Ningún control de calidad ni reporte de cobertura sobre los datos exportados.
- Formato (JSON) poco eficiente para cargas analíticas repetidas o para conectar con herramientas de BI.

Objetivos

Engineering (Data Engineering)

- Pipeline reproducible y orquestado desde un único punto de entrada.
- Transformación de datos raw a un modelo analítico (star schema).
- Controles de calidad de datos sobre la dimensión enriquecida.
- Persistencia en un formato columnar eficiente (Parquet).

Software (Backend / arquitectura)

- Separación de responsabilidades entre configuración, orquestación, ETL, enriquecimiento, validación y logging.
- Arquitectura mantenible y modular que permita incorporar nuevas dimensiones o etapas sin reescribir el núcleo.

Analytics

- Explorar patrones de escucha (temporales, por dispositivo, geográficos).
- Construir métricas reutilizables (tiempo de escucha, diversidad musical, consistencia de hábito).
- Generar insights interpretables a partir de esas métricas, tanto en BI como en el frontend.

3

Data Engineering

3.1 Fuentes de datos

- **Fuente:** exportación oficial del historial de reproducción de Spotify.
- **Formato:** archivos JSON, uno o varios por exportación.
- **Volumen:** variable según el historial de cada usuario; el proyecto soporta múltiples datasets en paralelo mediante la variable `SPOTIFY_USER_SUFFIX`, que separa las rutas de entrada y salida por usuario.
- **Estructura:** eventos de reproducción con timestamp (`ts`), metadata de track o episodio, plataforma/dispositivo, IP y país reportado por Spotify, y flags de comportamiento (`skipped` , `shuffle` , `offline` , `incognito_mode`).
- **Limitaciones conocidas:** columnas relacionadas con audiobooks frecuentemente vienen completamente nulas; la geolocalización nativa se limita a un código de país, sin ciudad, región ni ISP.

3.2 Arquitectura ETL

El pipeline separa responsabilidades en etapas. El orden real de ejecución orquestado por `main.py` es **Extract** → **Transform** → **Load** → **Enrich** → **Validate** — es decir, el reporte de calidad corre sobre la dimensión geográfica ya enriquecida, no antes de la carga a Parquet:

- **Extract:** lectura de todos los JSON del directorio configurado y unificación en una sola estructura.
- **Transform:** limpieza, construcción de dimensiones con claves subrogadas y construcción de la tabla de hechos.
- **Load:** persistencia de cada dimensión y de la tabla de hechos en Parquet.
- **Enrich:** consulta de las IPs únicas contra las bases MaxMind GeoLite2 para producir la dimensión geográfica enriquecida.
- **Validate:** reporte de calidad sobre esa dimensión enriquecida (no bloqueante).

3.3 Transformación de datos

- **Limpieza:** eliminación de columnas de audiobook completamente nulas; reinicio del índice.
- **Normalización de tipos:** uso de tipos de texto consistentes (`StringDtype`) para evitar conflictos en los merges entre tabla de hechos y dimensiones.
- **Duplicados:** cada dimensión deduplica sus atributos antes de asignar claves subrogadas.
- **Columnas derivadas:** `event_id` (secuencial, clave primaria de la tabla de hechos) y `date_id` (`YYYYMMDDHH` , derivado del timestamp).
- **Agregaciones:** a nivel de pipeline, las únicas agregaciones son descriptivas, dentro del reporte de calidad (top 10 ciudades, top 10 ISPs). Las agregaciones analíticas (rankings, métricas de comportamiento, crecimiento de biblioteca) se calculan más abajo en la cadena, sobre los Parquet ya publicados — ver sección 6.

3.4 Enriquecimiento de datos

Ejemplo concreto de enriquecimiento en este proyecto: **Spotify data + GeoIP (MaxMind GeoLite2) → dataset geográfico enriquecido.**

Las IPs públicas válidas se consultan contra `GeoLite2-City.mmdb` (ciudad, país, región, coordenadas) y `GeoLite2-ASN.mmdb` (organización autónoma / ISP). Las IPs privadas, reservadas, loopback, inválidas o desconocidas se omiten explícitamente antes de consultar. El resultado se combina con la dimensión de ubicación original y se persiste como una dimensión separada (`dim_location_enriched`), preservando el país originalmente reportado por Spotify.

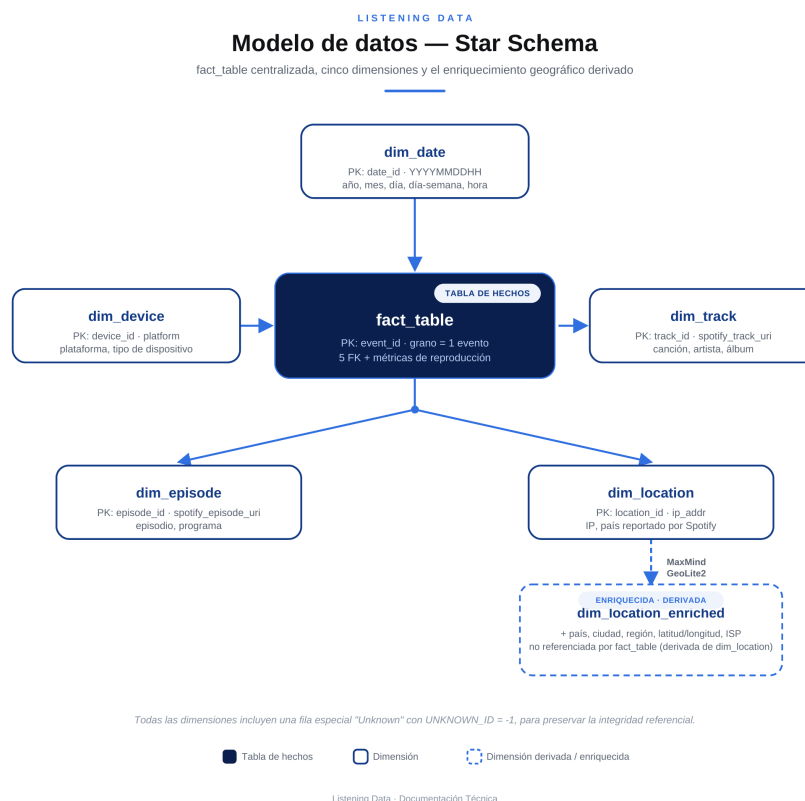
3.5 Calidad de datos

- **Schema checks:** tipado explícito y consistente en los merges (`StringDtype`, conversión explícita de columnas booleanas).
- **Null checks:** detección y eliminación de columnas completamente nulas (audiobooks); manejo explícito de valores desconocidos (`Unknown`, coordenadas `(0.0, 0.0)`) en vez de dejarlos como nulos sin marcar.
- **Uniqueness:** deduplicación de atributos en la construcción de cada dimensión.
- **Referential integrity:** todas las foreign keys de la tabla de hechos que no encuentran correspondencia en su dimensión reciben `UNKNOWN_ID = -1`, en vez de quedar nulas.
- **Validaciones / quality reports:** un módulo dedicado inspecciona la dimensión geográfica enriquecida y reporta volumen total, cobertura de atributos geográficos y los principales países/ISPs, con cada métrica protegida de forma independiente para que un fallo puntual no invalide el resto del reporte.
- **Tests:** cobertura mediante pruebas automatizadas con pytest sobre las distintas etapas del pipeline, sin sustituir la validación de datos anterior — ver sección 7.

4

Data Architecture & Modeling

4.1 Modelo de datos



En este proyecto, la tabla de hechos se llama `fact_table` (equivalente conceptual a lo que otros proyectos llaman `fact_stream`). **Grain:** una fila de `fact_table` representa un evento de reproducción individual, identificado por `event_id`.

Campo	Descripción
<code>event_id</code>	Primary key del evento
<code>date_id</code>	FK hacia <code>dim_date</code>
<code>track_id</code>	FK hacia <code>dim_track</code>
<code>episode_id</code>	FK hacia <code>dim_episode</code>
<code>device_id</code>	FK hacia <code>dim_device</code>
<code>location_id</code>	FK hacia <code>dim_location</code>
<code>ms_played</code>	Milisegundos reproducidos
<code>skipped / shuffle / offline / incognito_mode</code>	Flags booleanos de comportamiento

Dimensión	Clave	Clave natural	Atributos
dim_date	date_id	YYYYMMDDHH	Año, mes, día, día de semana, hora
dim_device	device_id	platform	Plataforma, tipo de dispositivo
dim_track	track_id	spotify_track_uri	Canción, artista, álbum
dim_episode	episode_id	spotify_episode_uri	Episodio, programa
dim_location	location_id	ip_addr	IP, país reportado por Spotify; enriquecida con país, ciudad, región, ISP, latitud y longitud

Todas las dimensiones usan surrogate keys enteras y secuenciales, más una fila especial **Unknown** con **UNKNOWN_ID = -1**, lo que permite mantener la integridad referencial del modelo aun con datos faltantes.

4.2 ¿Por qué Star Schema?

El esquema estrella surge naturalmente de la estructura del dato de origen: cada registro JSON representa una reproducción, es decir, un evento analítico que puede convertirse directamente en un hecho. Sus atributos describen tanto métricas de la reproducción como referencias a distintas dimensiones (fecha, dispositivo, pista, ubicación, contenido). A partir de esta estructura se separó la información descriptiva en dimensiones reutilizables y se mantuvo la reproducción como tabla de hechos central. Esto permite analizar el historial desde distintas perspectivas sin alterar la granularidad original del dato, manteniendo un modelo simple y directamente utilizable mediante SQL y herramientas de BI como Power BI o Tableau.

4.3 Almacenamiento

Todos los datasets se persisten en **Apache Parquet**:

- **Columnar**: permite leer solo las columnas necesarias para cada consulta, en vez de la fila completa.
- **Compresión**: el formato aplica compresión columnar, reduciendo el tamaño en disco frente a JSON o CSV equivalentes.
- **Schema explícito**: los tipos se fijan durante la escritura, evitando la ambigüedad de tipos típica de JSON.
- **Eficiencia analítica**: formato nativo para motores columnar como DuckDB, usado por el frontend.
- **Compatibilidad con BI**: Power BI y otras herramientas con conectores Parquet pueden leer estos archivos sin transformación adicional.

5

Backend Engineering

5.1 Software Architecture

El backend del proyecto se estructura como un pipeline de procesamiento de datos modular, donde cada responsabilidad se encuentra separada en componentes independientes. El punto de entrada (`main.py`) coordina la ejecución del flujo, mientras que los módulos de `src/` encapsulan las distintas etapas de procesamiento.

La arquitectura separa principalmente:

- **Orquestación:** coordinación de las etapas y configuración de la ejecución.
- **Extracción y transformación:** lectura del historial de Spotify, limpieza y normalización de los registros.
- **Modelado:** construcción de dimensiones y tabla de hechos a partir de los eventos de reproducción.
- **Enriquecimiento:** incorporación de información geográfica a partir de las direcciones IP.
- **Validación:** comprobación de integridad y calidad de los datos generados.
- **Persistencia:** generación de archivos Parquet como formato final de almacenamiento analítico.
- **Logging y utilidades:** funcionalidades transversales utilizadas por los distintos módulos.

Esta separación permite que cada componente tenga una responsabilidad acotada y pueda evolucionar o probarse de forma independiente, evitando concentrar toda la lógica de transformación en un único script.

5.2 Data Processing Pipeline

La ejecución del backend sigue un flujo determinista desde los archivos JSON originales hasta el dataset analítico final:

```
JSON de Spotify → extracción → transformación → enriquecimiento → validación → modelo dimensional → Parquet
```

El pipeline mantiene una separación clara entre los datos de entrada, las transformaciones intermedias y los artefactos finales. Esto permite reproducir el proceso completo a partir del dataset fuente y facilita identificar en qué etapa se produce una modificación o validación específica.

La configuración se centraliza en un único módulo, donde se definen rutas, nombres de archivos y constantes de dominio. Además, `SPOTIFY_USER_SUFFIX` permite ejecutar el mismo pipeline sobre distintos datasets sin modificar la lógica de procesamiento.

5.3 Data Contracts

Aunque el proyecto trabaja principalmente con archivos Parquet en lugar de una API tradicional, existen contratos explícitos en las fronteras entre las distintas capas del sistema.

En el backend, estos contratos se manifiestan en la estructura definida para las dimensiones y la tabla de hechos, así como en los esquemas esperados durante las distintas etapas del pipeline.

En el frontend, las interfaces TypeScript representan explícitamente las estructuras de los resultados obtenidos mediante SQL. Esto permite mantener tipados los datos desde la consulta hasta los componentes de visualización y evita depender de estructuras implícitas o valores sin tipo.

De esta forma, el modelo de datos actúa como contrato compartido entre el procesamiento, la consulta y la visualización.

5.4 Data Quality & Testing

La calidad del pipeline se valida mediante pruebas automatizadas con **pytest**. Las pruebas cubren transformaciones, reglas de procesamiento y componentes relevantes del pipeline, permitiendo detectar regresiones cuando se modifica la lógica.

La suite mantiene una cobertura superior al **95%**, utilizando `pytest-cov` para medir qué proporción del código ejecutable se encuentra cubierta por pruebas.

La validación no se limita a comprobar que el pipeline termine correctamente: también se verifican condiciones esperadas sobre los datos generados, de modo que un proceso exitoso técnicamente no implique necesariamente un dataset incorrecto.

5.5 Backend–Frontend Integration

El frontend utiliza **DuckDB WASM** como motor de consulta sobre los archivos Parquet generados por el pipeline. De esta manera, la capa de procesamiento y la capa de visualización mantienen una separación clara: el backend produce los datos analíticos y el frontend ejecuta las consultas necesarias para construir las visualizaciones.

El flujo de consumo es:

```
Pipeline Python → Parquet → DuckDB WASM → SQL → TypeScript → componentes React
```

Esta arquitectura permite conservar la expresividad de SQL para el análisis, mantener los datos en un formato columnar eficiente y entregar resultados tipados a la capa de presentación.

5.6 Engineering Practices

El desarrollo del backend aplica prácticas orientadas a mantenibilidad y reproducibilidad:

- **Modularización:** responsabilidades distribuidas entre módulos especializados.
- **Configuración centralizada:** rutas y parámetros de dominio definidos en un único punto.
- **Tipado:** estructuras de datos explícitas en las interfaces entre capas.
- **Logging:** trazabilidad de la ejecución del pipeline y sus distintas etapas.
- **Testing automatizado:** pruebas mediante pytest y medición de cobertura.
- **Reproducibilidad:** generación determinista de los artefactos analíticos a partir de los datos fuente.
- **Separación de responsabilidades:** procesamiento, validación, enriquecimiento y persistencia desacoplados.

6

Frontend & Data Visualization

El frontend se presenta como un **Data Product en sí mismo** — una forma de responder preguntas analíticas — y no solo como una interfaz.

Propósito

Preguntas que el usuario puede responder directamente desde la interfaz: cuánto escucha, cuándo escucha, qué escucha, qué tan diversa es su escucha, y qué tan consistente es su hábito de escucha en el tiempo.

Diseño de visualización

El patrón usado para diseñar cada visualización es:

Question → Metric → Visualization → Insight

Question	Metric	Visualization	Insight
¿Cómo varía mi escucha en el tiempo?	Horas por año	Línea / área	Tendencia temporal general
¿Qué artistas dominan mi escucha?	Eventos y horas por artista (top 10 + tabla completa)	Barras horizontales	Concentración de favoritos vs. cola larga
¿Cuándo escucho más en la semana?	Eventos por día y hora	Heatmap	Patrón horario de comportamiento
¿Cómo varía mi comportamiento por dispositivo?	Eventos, horas y duración media por dispositivo	Barras + línea	Diferencias de hábito según contexto
¿Qué tan diversa es mi biblioteca?	Artistas únicos / eventos totales	Indicador de proporción	Amplitud vs. repetición
¿Cómo creció mi biblioteca?	Nuevos artistas/tracks/álbumes por año	Línea múltiple	Ritmo de descubrimiento musical

Cada visualización incluye estados explícitos de carga, error y ausencia de datos, y un visor de solo lectura que muestra el SQL exacto que produjo esa visualización — lo que hace trazable la relación entre pregunta, métrica y gráfico.

7

Testing & Data Quality

La confiabilidad del pipeline no depende solo de que la ejecución termine sin errores: se valida activamente mediante una suite de tests automatizados y controles de calidad sobre los datos que efectivamente se generan.

La suite está organizada por módulo y por tipo de prueba, cubriendo tanto unidades de lógica aisladas como el flujo de integración completo:

- **Unit tests:** por módulo (ReadProcess, localization, validate, utils), cubriendo funciones individuales de extracción, transformación, enriquecimiento y validación.
- **Data quality tests:** validan el comportamiento del reporte de calidad y el manejo de valores desconocidos/faltantes.
- **Integration tests:** un test end-to-end cubre el flujo completo orquestado por `main.py`, desde los JSON de entrada hasta los Parquet finales.
- **Frontend tests:** no se documentan tests automatizados para la capa de frontend en las fuentes disponibles.
- **Fixtures y mocks:** DataFrames mínimos, IPs de rangos reservados para documentación, directorios temporales y un mock del lector de GeoLite2 con atributos primitivos explícitos, para evitar problemas de serialización con PyArrow.

Estas pruebas se ejecutan con pytest y se apoyan en fixtures reutilizables (vía `conftest.py`) para mantener los casos de prueba livianos y deterministas, sin depender de datos reales ni de las bases GeoLite2 completas.

COVERAGE

La cobertura de tests confirmada, medida con `pytest-cov`, es **superior al 95%** del código ejecutable del pipeline.

8

Engineering Decisions / ADRs

ADR-1

Apache Parquet como formato de persistencia

DECISION

Usar Apache Parquet para todos los datasets generados por el pipeline.

CONTEXT

Se necesita un formato eficiente para cargas de trabajo analíticas, compatible con herramientas de BI y con motores de consulta embebidos.

OPTIONS

CSV, JSON, una base de datos relacional.

RATIONALE

Carga de trabajo predominantemente analítica; almacenamiento columnar; compresión; schema explícito y tipado.

TRADE-OFFS

Menos apropiado que una base de datos relacional para cargas de trabajo transaccionales (escrituras frecuentes, actualizaciones puntuales).

ADR-2

Star Schema en vez de un modelo normalizado

DECISION

Modelar los datos como esquema estrella (una tabla de hechos + dimensiones desnormalizadas).

CONTEXT

El consumo principal es analítico (BI, exploración interactiva), no transaccional.

OPTIONS

Un modelo normalizado (3NF), una única tabla ancha ("long format") sin dimensiones separadas.

RATIONALE

Dimensiones reutilizables entre consultas, menos joins para preguntas analíticas típicas, modelo fácil de entender tanto para BI como para el motor SQL del frontend.

TRADE-OFFS

Requiere una etapa de ETL explícita para construir las dimensiones; menos flexible si en el futuro se necesitaran actualizaciones transaccionales sobre los datos.

ADR-3

DuckDB WASM como motor de consultas en el navegador

DECISION

Ejecutar las consultas SQL directamente en el navegador mediante DuckDB WASM, sobre los archivos Parquet publicados.

CONTEXT

Se necesita una capa de exploración interactiva capaz de ejecutar consultas analíticas SQL completas sobre el modelo dimensional, con buen rendimiento y despliegue simple.

OPTIONS

Un motor de consultas gestionado en un servicio externo; procesar los datos en el cliente con lógica ad-hoc en JavaScript en vez de SQL.

RATIONALE

DuckDB WASM permite ejecutar SQL completo directamente sobre Parquet dentro del navegador, con muy buen rendimiento para el tamaño de este dataset, desplegable como sitio estático y sin infraestructura adicional que mantener.

TRADE-OFFS

El rendimiento depende del cómputo del cliente; los Parquet publicados son accesibles por cualquiera con el enlace, aunque la interfaz no expone las columnas sensibles (IP, ISP, coordenadas exactas).

ADR-4

UNKNOWN_ID = -1 en vez de valores nulos en las foreign keys

DECISION

Usar un valor centinela (-1) para representar relaciones sin correspondencia dimensional, en vez de dejar la foreign key nula.

CONTEXT

Se necesita preservar la integridad referencial del modelo incluso cuando faltan datos (por ejemplo, un dispositivo o ubicación no identificable).

OPTIONS

Dejar la FK como NULL; usar 0 como valor centinela.

RATIONALE

-1 es inequívocamente distinto de cualquier surrogate key real, lo que evita ambigüedad en los joins y en las herramientas de BI que no siempre manejan bien los NULL en claves.

TRADE-OFFS

Es una convención específica del proyecto: cualquier consumidor nuevo de los datos debe conocerla explícitamente para no interpretar -1 como un valor real.

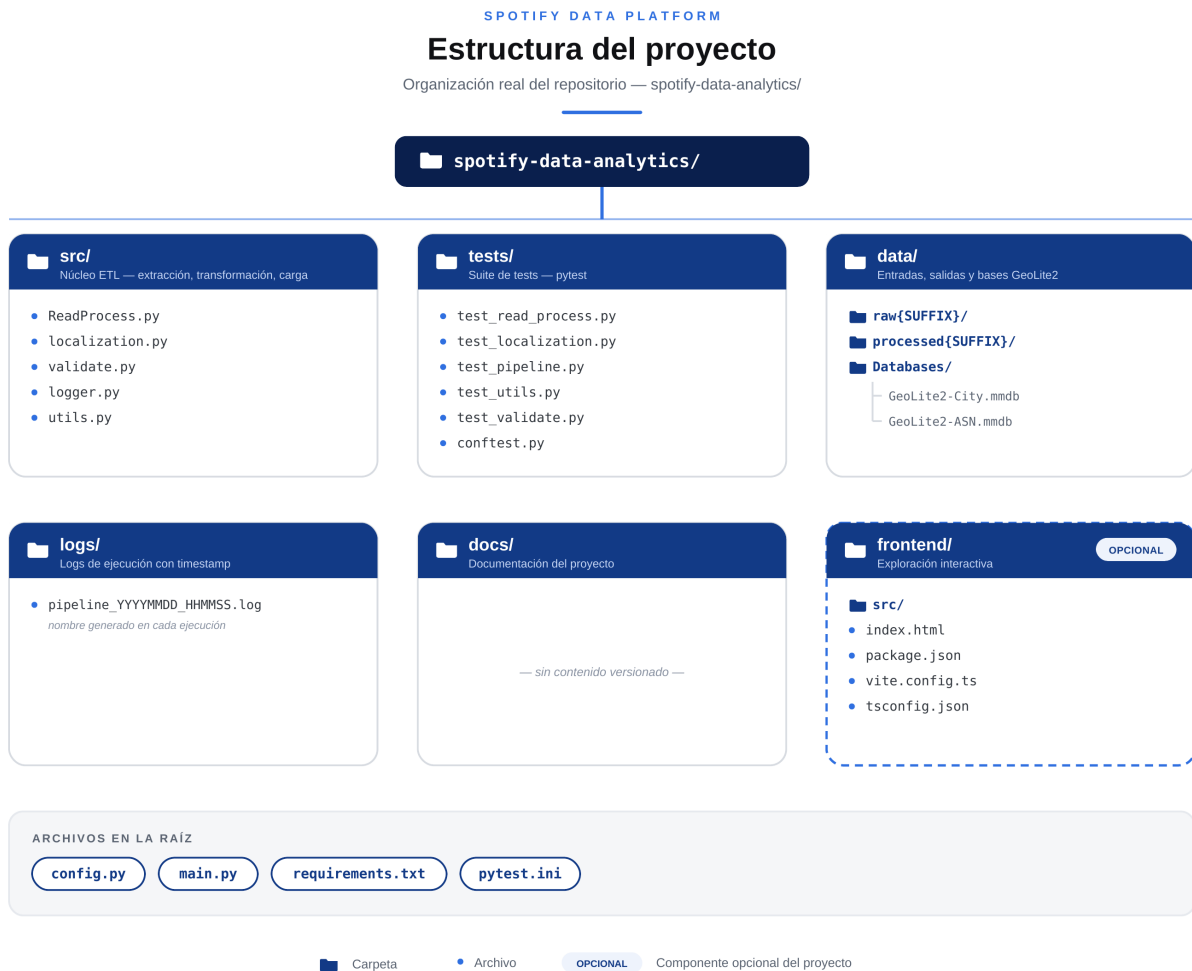
9

Reproducibility

- **Requisitos:** Python 3.x y las dependencias del pipeline (pandas, numpy, PyArrow, GeolIP2, pytest y su ecosistema, python-dotenv); para el frontend, un entorno Node.js con las dependencias declaradas en su propio manifiesto (React, TypeScript, Vite, DuckDB WASM, ECharts, entre otras).
- **Variables de entorno:** `SPOTIFY_USER_SUFFIX` es opcional y permite separar datasets por usuario; si no se define, el pipeline usa las rutas por defecto.
- **Instalación:** instalar las dependencias Python declaradas para el pipeline, y por separado las dependencias del frontend, si se desea ejecutar esa capa.
- **Generación de datos:** el historial de reproducción debe solicitarse y exportarse desde la cuenta de Spotify del usuario; las bases MaxMind GeoLite2 (City y ASN) deben obtenerse por separado según la licencia de MaxMind, ya que no se distribuyen junto con el repositorio.
- **Ejecución del ETL:** se ejecuta a través del punto de entrada único del pipeline, que orquesta las fases descritas en la sección 3.2.
- **Tests:** la suite de pytest cubre ETL, enriquecimiento, validación, utilidades y el flujo de integración completo, con soporte para aislar tests unitarios de integración y para generar reportes de cobertura.
- **Ejecución del frontend:** durante desarrollo se sirve mediante el servidor de desarrollo de Vite; para producción se genera un build estático que puede servirse como cualquier sitio estático, ya que no depende de un servidor propio.

10

Project Structure



Spotify Data Platform · Documentación Técnica

Esta estructura refleja el repositorio real documentado. Difiere intencionalmente de una plantilla genérica que organizaría el ETL en subcarpetas por etapa (`etl/extract`, `etl/transform`, `etl/enrich`, `etl/quality`): en este proyecto la lógica ETL vive dentro de `src/`, organizada por módulo de responsabilidad en vez de por carpeta de etapa.